Name : Bhoomi Mangesh Naik
Class : D15C
Roll No. : 34
Practical No. : 9

Aim : Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

Theory :

1. Dataset Source

The dataset used for this experiment is obtained from Kaggle:
IMDB Dataset of 50K Movie Reviews
https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

2. Dataset Description

The IMDB dataset contains movie reviews labeled as positive or negative, used for sentiment analysis tasks in natural language processing.

Dataset Characteristics

Total Records: 50,000 reviews (25,000 training + 25,000 testing)
Type: Text dataset
Learning Type: Supervised learning (Binary Classification)

Features Used

Feature	Description
Review Text	Movie review written in natural language
Sentiment	Label (positive / negative)

3. Mathematical Formulation

3.1 Recurrent Neural Network (RNN)

RNN is designed to process sequential data by maintaining a hidden state that captures information from previous inputs.

Hidden State Update:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

3.2 Long Short-Term Memory (LSTM)

- LSTM is an advanced RNN that solves the vanishing gradient problem using gates.

Forget Gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input Gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Cell State Update:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Final Hidden State:

$$h_t = o_t * \tanh(C_t)$$

3.3 Fully Connected Layer

After sequence processing, output is passed to dense layers:

y=f(Wx+b)

3.4 Loss Function

For binary classification, binary cross-entropy is used:

$$Loss = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

3.5 Backpropagation Through Time (BPTT)

Weights are updated using gradient descent:

$$\boxed{W = W - \eta \frac{\partial L}{\partial W}}$$

Where:

- η = learning rate
- $\boxed{\frac{\partial L}{\partial W}}$ = gradient of loss

4. Algorithm Limitations

RNN / LSTM Limitations

- Vanishing gradient problem (in simple RNN)
- High computational complexity
- Training time is longer for long sequences
- Requires large labeled text data
- LSTM models are memory intensive
- Difficult to interpret model decisions
- May overfit without proper regularization

5. Methodology / Workflow

Dataset collection from Kaggle
Data upload in Google Colab
Text preprocessing:

- Tokenization
- Stopword removal
- Padding sequences

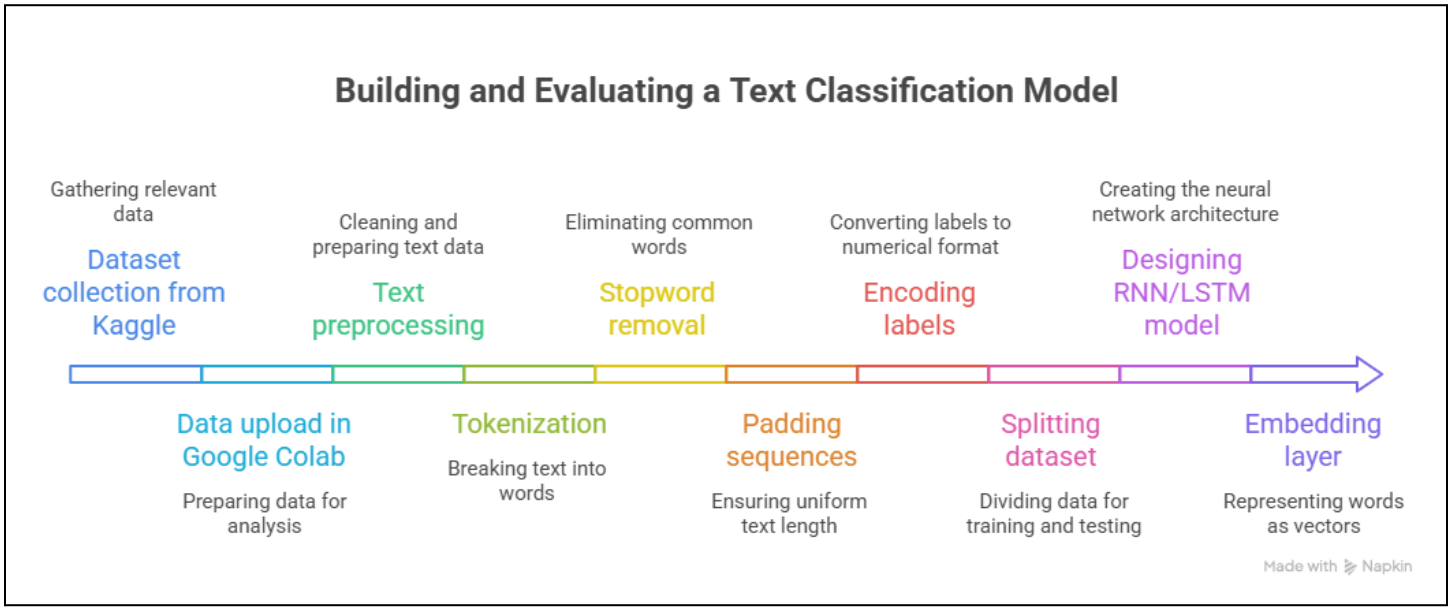
Encoding labels (positive = 1, negative = 0)
Splitting dataset into training and testing sets

Designing RNN/LSTM model using Keras/TensorFlow:

- Embedding layer
- RNN or LSTM layers
- Dense layers

Compiling model (optimizer, loss function)
Training the model (fit function)
Evaluating model performance
Prediction on test data

Workflow Diagram:



6. Performance Analysis

- Evaluated using accuracy and loss metrics
- Training vs validation accuracy helps detect overfitting
- Loss curve shows model convergence
- Confusion matrix used for classification performance
- High accuracy indicates effective feature extraction by CNN

7. Hyperparameter Tuning

CNN Hyperparameters

- Number of convolutional layers
- Number of filters (kernels)
- Kernel size
- Pooling size
- Learning rate
- Batch size
- Number of epochs
- Activation functions
- Dropout rate

Impact:

- Improves model accuracy and generalization
- Helps in better feature extraction
- Reduces overfitting and underfitting
- Enhances prediction performance

Code and Output :

```
from google.colab import files
uploaded = files.upload()
import pandas as pd
df = pd.read_csv('IMDB Dataset.csv')

df.head()
df.info()
df['sentiment'].value_counts()
# Convert sentiment to binary
df['sentiment'] = df['sentiment'].map({'positive':1, 'negative':0})

df.head()
import re
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = re.sub(r'<.*?>', '', text)   # remove HTML tags
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    text = text.lower()
    text = " ".join([word for word in text.split() if word not in stop_words])
    return text

df['review'] = df['review'].apply(clean_text)
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

max_words = 10000
max_len = 200

tokenizer = Tokenizer(num_words=max_words)
```

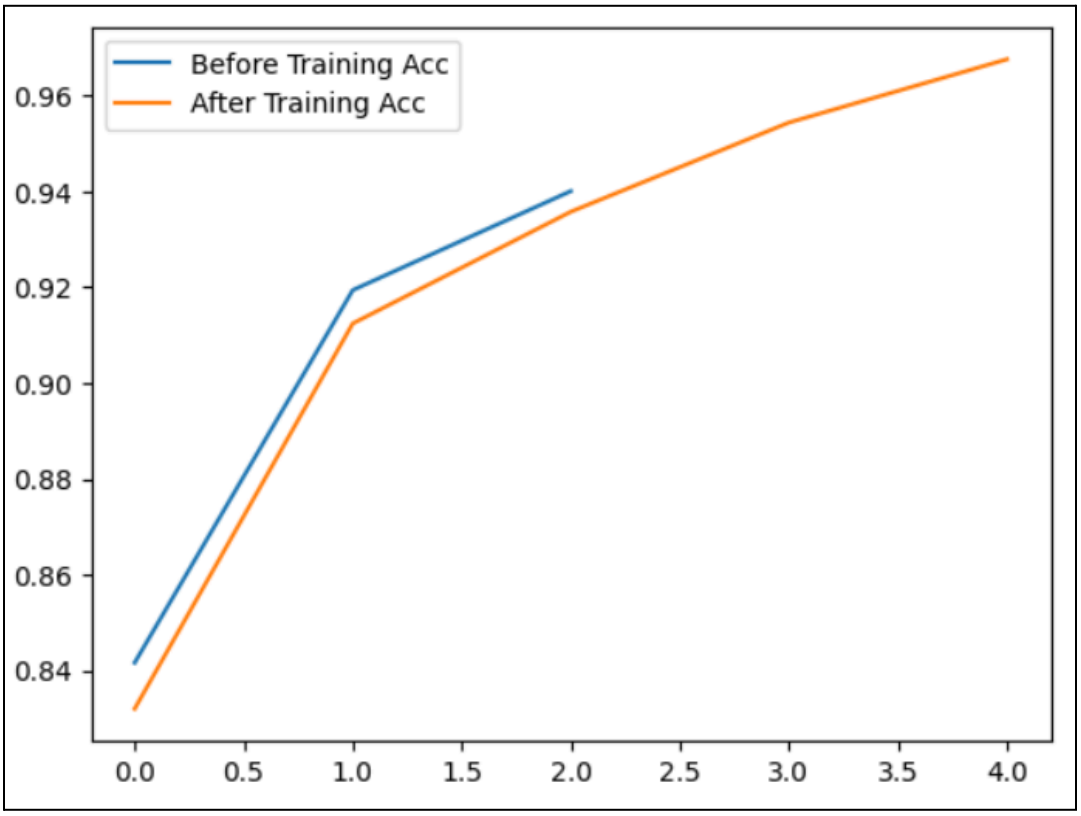
```
tokenizer.fit_on_texts(df['review'])

X = tokenizer.texts_to_sequences(df['review'])
X = pad_sequences(X, maxlen=max_len)

y = df['sentiment']
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

model = Sequential([
    Embedding(input_dim=max_words, output_dim=128, input_length=max_len),
    LSTM(64),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
history = model.fit(
    X_train, y_train,
    epochs=3,
    batch_size=128,
    validation_split=0.2
)
loss, acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", acc)
from tensorflow.keras.layers import Dropout, Bidirectional

model_tuned = Sequential([
    Embedding(input_dim=max_words, output_dim=128, input_length=max_len),

    Bidirectional(LSTM(128, return_sequences=True)),
    Dropout(0.3),
```

```
    LSTM(64),
    Dropout(0.3),

    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_tuned.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model_tuned.summary()
history_tuned = model_tuned.fit(
    X_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
loss_tuned, acc_tuned = model_tuned.evaluate(X_test, y_test)

print("Accuracy Before Tuning:", acc)
print("Accuracy After Tuning:", acc_tuned)
sample_review = ["This movie was absolutely amazing and thrilling"]

sample_seq = tokenizer.texts_to_sequences(sample_review)
sample_pad = pad_sequences(sample_seq, maxlen=max_len)

prediction = model_tuned.predict(sample_pad)

print("Prediction:", "Positive" if prediction[0][0] > 0.5 else "Negative")
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Before Training Acc')
plt.plot(history_tuned.history['accuracy'], label='After Training Acc')
plt.legend()
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 14s 44ms/step - accuracy: 0.8783 - loss: 0.3017
Test Accuracy: 0.8783000111579895
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 107s 337ms/step - accuracy: 0.8623 - loss: 0.4557
Accuracy Before Tuning: 0.8783000111579895
Accuracy After Tuning: 0.8622999787330627
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 852ms/step
Prediction: Positive
```



Google Colab Link for Code and Output : [Link for Code and Output]

Conclusion :

The RNN/LSTM model was successfully implemented for sentiment analysis on the IMDB dataset. The model effectively learned sequential patterns in text data, enabling accurate classification of reviews as positive or negative. The use of LSTM helped overcome the vanishing gradient problem and improved performance. Training and validation results showed good convergence with satisfactory accuracy. Thus, RNN/LSTM models are highly effective for processing and analyzing sequential text data.